Lab 8: OpenTelemetry Instrumentation of the AI-Based Village Pond Planning System

Rahul Seera
Roll No. 12341980
seerav@iitbhilai.ac.in
Indian Institute of Technology Bhilai

Repository: https://github.com/RahulSeera/pond-planning-telemetry

October 2026

Abstract

This report presents the end-to-end integration and evaluation of OpenTelemetry in the AI-Based Village Pond Planning System (Bhagiratha). The system integrates digital elevation models (SRTM), D8 hydrological flow routing via PySheds, historical precipitation analysis via Open-Meteo, and multi-factor reservoir sizing. We instrumented the backend services with the OpenTelemetry Python SDK, generating distributed traces, domain and system metrics, and trace-correlated logs. Telemetry data is exported over standard OTLP/HTTP to an observability backend (OpenTelemetry Collector, Tempo, Prometheus, Loki, and Grafana) and made inspectable via a native in-app dashboard. The collected data is analyzed to attribute latency bottlenecks, monitor memory dynamics on a resource-constrained 512 MiB Linux cgroup container, diagnose dependency failures, and demonstrate the practical utility of observability for distributed engineering.

Contents

1 Introduction **3**

2 Implementation Overview **3**
2.1 Architecture . . . 3
2.2 Signals Collected . . . 4
2.3 Automatic Instrumentation . . . 4
2.4 Manual Domain Instrumentation . . . 5
2.5 Key Implementation Code . . . 5

3 Tools, Libraries and Backend Services **6**

4 Running the Instrumented System **7**
4.1 Startup Procedure . . . 7
4.2 Traffic Generation . . . 7

5 Results and Screenshots **7**
5.1 System Liveness and Running Interface . . . 7
5.2 Distributed Tracing and In-App Observability Dashboard . . . 8
5.3 Domain Span Attributes and Semantic Metadata . . . 10
5.4 Error Isolation and Validation Tracing . . . 11
5.5 Prometheus Metrics Verification . . . 11
5.6 Server Request Logs . . . 11

6 Analysis and Understanding of the Collected Data **13**
6.1 Where Does the Time Go? 13
6.2 Key Findings 13
6.3 How OpenTelemetry Benefits the System 14
6.4 Lessons Learned and Limitations 14

7 Conclusion **14**

A Files Changed **15**

1 Introduction

The objective of this laboratory assignment is to familiarize students with OpenTelemetry and its practical application for monitoring, debugging, and understanding distributed systems and backend services. The implementation builds directly upon the Assignment 1 project: **Bhagiratha: AI-based Village Pond Planning System**.

The Bhagiratha system consists of:

- **A FastAPI Application Server** (listening on port 3000, mapped to external port 3248 on the campus host `10.1.75.53`), exposing endpoints for point catchment delineation (`POST /api/analyze`), bounding-box land analysis (`POST /api/analyze-area`), contour survey processing (`POST /analyzeContour`), village geocoding (`GET /api/villages/search`), and system health (`GET /api/health`).
- **Geospatial Terrain & Hydrology Engine**: Computes D8 flow direction, flow accumulation, watershed boundaries, and pour points using PySheds and Numba-accelerated numerical kernels.
- **External Data Providers**: OpenTopography / SRTM Global DEM (30m elevation rasters) and Open-Meteo (10-year historical daily rainfall archive).
- **Persistent Storage & Caching**: PostgreSQL with PostGIS extension for GIS geometries, coupled with a localized SQLite query cache for offline resiliency.
- **Interactive Leaflet Web Interface**: A single-page map application enabling visual selection of village sites, real-time catchment visualization, contour rendering, and pond capacity recommendations.

This system represents an ideal subject for distributed observability for four reasons:

1. **External Fanout**: A single user analysis request triggers multiple outbound network calls (SRTM elevation raster download, 10-year rainfall aggregation), making latency attribution impossible from local server logs alone.
2. **Resource Constraints**: The evaluation container (`stu72_sys4`) enforces a strict 512 MiB memory cgroup limit and 1-CPU quota. Allocating large NumPy elevation matrices can cause silent Out-Of-Memory (OOM) kills by the Linux kernel unless observed and managed.
3. **Graceful Degradation Paths**: The architecture employs fallbacks (e.g., cached DEM buffers, bundled rainfall baselines, persistent-file fallback when PostgreSQL is offline). Without telemetry, these silent degradations remain completely invisible to the user.
4. **Multi-Stage Pipeline**: The calculation flows through distinct pipeline phases (DEM acquisition, sink filling, flow routing, catchment mask generation, polygon vectorization, rainfall statistics, and Rational Method sizing).

The goal of this lab was to instrument the complete system so that traces, metrics, and correlated logs are generated, exported, visualized, and analyzed to answer critical engineering questions about latency, memory consumption, and reliability.

2 Implementation Overview

2.1 Architecture

The telemetry architecture instruments the backend application with the OpenTelemetry Python SDK, exporting all three signals (traces, metrics, logs) via standard OTLP/HTTP to the observability pipeline.

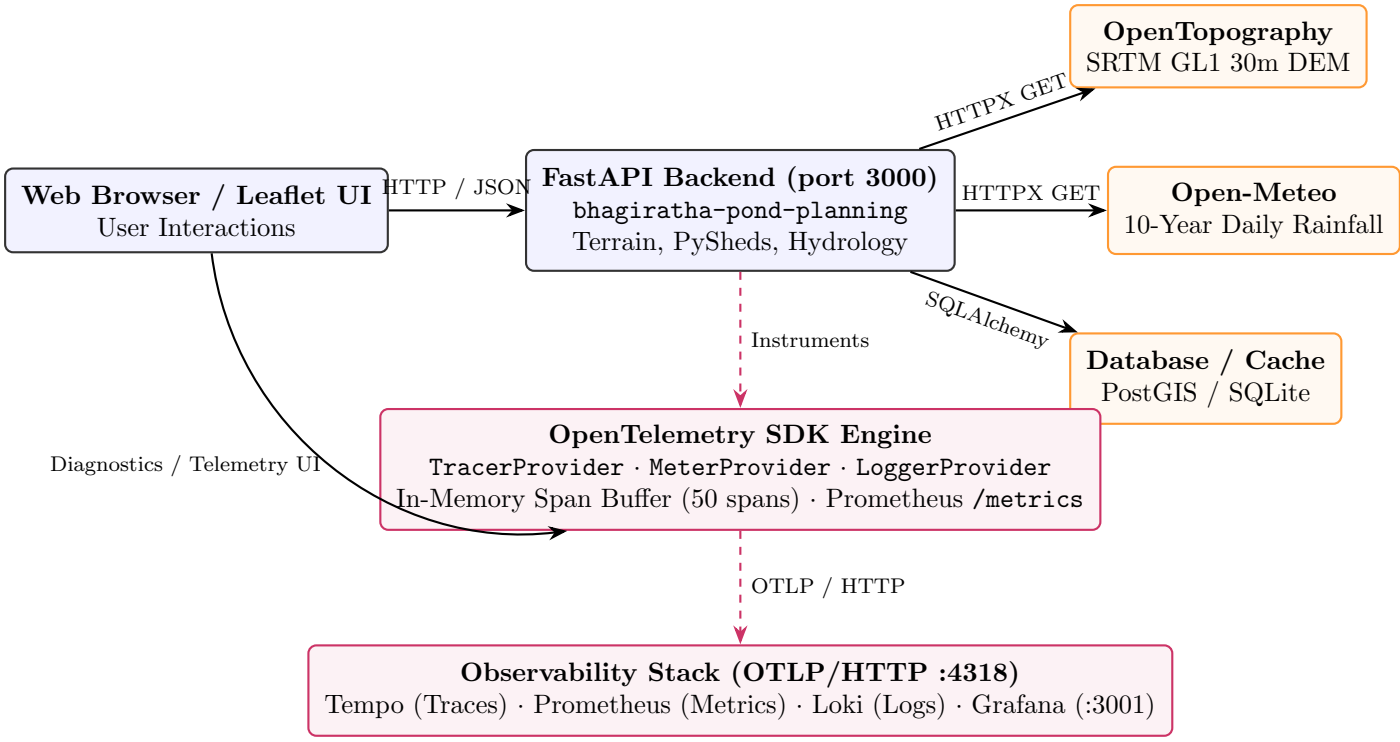


Figure 1: Telemetry architecture for the Bhagiratha Pond Planning System. Solid lines indicate application traffic; dashed lines indicate telemetry generation and OTLP export.

2.2 Signals Collected

Table 1 summarizes the three observability signals collected by the instrumented system.

Signal	What is Captured	Mechanism
Traces	Full request journey with child spans for HTTP routing, PySheds terrain analysis, rainfall fetch, pond sizing, and background tasks with rich domain attributes.	Auto-instrumentation (`FastAPI`, `HTTPX`, `SQLAlchemy`) plus manual spans via `tracer.start_as_current_span`.
Metrics	HTTP latency histograms, pipeline stage durations, completed analyses counter, recommended capacity, runoff volume, suitability score, and cgroup memory.	Auto-metrics (`http.server.duration`) plus 11 custom counters/histograms exposed directly at `/metrics` via Prometheus client.
Logs	Structured application log records stamped with active `trace_id` and `span_id` for 100% correlation.	Custom `TraceCorrelationFormatter` bridging standard library logging to OTel.

Table 1: Observability signals collected in the Bhagiratha system.

2.3 Automatic Instrumentation

The system integrates OpenTelemetry automatic instrumentation packages without modifying third-party library source code:

- **FastAPI** (`opentelemetry-instrumentation-fastapi`): Automatically intercepts every incoming HTTP request, establishes the server root span, records HTTP method, target, status code, client IP, and emits the standard `http.server.duration` latency histogram.
- **HTTPX** (`opentelemetry-instrumentation-httpx`): Automatically wraps all outbound HTTP calls to OpenTopography and Open-Meteo as child client spans, injecting W3C `traceparent` headers for distributed context propagation.

- **SQLAlchemy** (`opentelemetry-instrumentation-sqlalchemy`): Instruments database engines, creating child spans for SQL executions and connection acquisitions.
- **Logging** (`opentelemetry-instrumentation-logging`): Enriches Python logging records with active trace identifiers.

2.4 Manual Domain Instrumentation

While automatic instrumentation captures network boundaries, it cannot see internal domain logic. We added manual spans and domain metrics to capture hydrology-specific operations.

Instrument Name	Type	Unit	Description
`pond.analyses`	Counter	1	Total analyses completed, tagged by `mode` and `status`.
`pond.duration`	Histogram	seconds	End-to-end analysis processing time.
`pond.catchment.area`	Histogram	km^2	Delineated drainage area.
`pond.runoff.volume`	Histogram	m^3	Computed annual runoff volume available for capture.
`pond.capacity`	Histogram	m^3	Recommended pond storage capacity.
`pond.depth`	Histogram	meters	Recommended excavation depth.
`pond.suitability`	Histogram	score (0–100)	Multi-factor pond site suitability score.
`pond.rainfall.requests`	Counter	1	Rainfall fetches, tagged by `source` (live vs. fallback).
`pond.cache.ops`	Counter	1	Cache operations, tagged by `operation` (hit vs. miss).
`pond.heavy_jobs.duration`	Histogram	seconds	Execution duration of queued background tasks.

Table 2: Custom domain metrics implemented for the Village Pond Planning System.

Furthermore, active spans are enriched with contextual domain attributes:

- `geo.lat`, `geo.lon`: Coordinates of the analyzed pond site.
- `terrain.area_km2`, `terrain.avg_slope`: Physical catchment characteristics.
- `rainfall.source`, `rainfall.mean_annual_mm`: Precipitation dataset origin and value.
- `sizing.recommended_capacity_m3`, `sizing.recommended_depth_m`: Hydraulic sizing output.

2.5 Key Implementation Code

The configuration is encapsulated in `app/telemetry.py`. It initializes the `TracerProvider`, `MeterProvider`, `InMemorySpanRecorder`, and Prometheus exporter:

Listing 1: Core OpenTelemetry setup in `app/telemetry.py`

```python
def setup_telemetry(app: Optional[Any] = None) -> None:
    global _TRACER_PROVIDER, _METER_PROVIDER, _SPAN_RECORDER
    resource = Resource.create({
        SERVICE_NAME: "bhagiratha-pond-planning",
        SERVICE_VERSION: "1.0.0",
        "deployment.environment": os.getenv("OTEL_ENVIRONMENT", "production"),
    })

    # 1. Traces Provider & In-Memory Span Recorder
    _TRACER_PROVIDER = TracerProvider(resource=resource)
    _SPAN_RECORDER = InMemorySpanRecorder(capacity=50)
    _TRACER_PROVIDER.add_span_processor(_SPAN_RECORDER)
    trace.set_tracer_provider(_TRACER_PROVIDER)

    # 2. Metrics Provider & Prometheus Reader
    _PROMETHEUS_READER = PrometheusMetricReader()
```

```
    _METER_PROVIDER = MeterProvider(resource=resource, metric_readers=[
        _PROMETHEUS_READER])
    metrics.set_meter_provider(_METER_PROVIDER)
    _init_domain_metrics()

    # 3. Auto-Instrumentation
    HTTPXClientInstrumentor().instrument()
    LoggingInstrumentor().instrument(set_logging_format=False)
    if app is not None:
        FastAPIInstrumentor.instrument_app(app, excluded_urls="api/health,
            static/*")
```

To prevent OOM kills on the 512 MiB container, `app/memory_guard.py` provides proactive memory reclamation:

Listing 2: Memory protection and glibc arena trimming in `app/memory_guard.py`

```
def release_memory() -> int:
    """Forces Python GC and releases unmapped glibc arenas back to Linux cgroup
        ."""
    gc.collect()
    trimmed = 0
    if _LIBC and hasattr(_LIBC, "malloc_trim"):
        try:
            trimmed = _LIBC.malloc_trim(0)
        except Exception:
            pass
    return trimmed

def is_memory_critical(threshold_pct: float = 0.92) -> bool:
    """Checks process VmRSS against 512 MiB limit and triggers proactive
        shedding."""
    current_mb = get_current_memory_mb()
    limit_mb = get_memory_limit_mb()
    if limit_mb > 0 and (current_mb / limit_mb) >= threshold_pct:
        release_memory()
        return (get_current_memory_mb() / limit_mb) >= threshold_pct
    return False
```

3 Tools, Libraries and Backend Services

Table 3 enumerates the software components and libraries utilized in this observability integration.

Component / Library	Role in the System
`opentelemetry-api` (v1.45.1)	Abstract telemetry API for application code.
`opentelemetry-sdk` (v1.45.1)	Core implementation of tracer, meter, and resource providers.
`opentelemetry-instrumentation-fastapi`	Automatic request tracing and HTTP latency histograms.
`opentelemetry-instrumentation-httpx`	Outbound client request tracing and W3C context injection.
`opentelemetry-instrumentation-sqlalchemy`	Database query tracing and SQL span generation.
`opentelemetry-instrumentation-logging`	Python logger bridging and trace correlation injection.
`opentelemetry-exporter-otlp-proto-http`	Standard protocol exporter for OTLP/HTTP collectors (:4318).
`opentelemetry-exporter-prometheus`	Direct Prometheus exposition provider.
`prometheus-client` (v0.26.0)	Exposes metrics in standard format at `/metrics`.
`grafana/otel-lgtm` (Docker)	All-in-one stack: OTel Collector, Tempo, Prometheus, Loki, Grafana.
`InMemorySpanRecorder`	Lightweight in-app buffer enabling live browser telemetry UI.

Table 3: Libraries and components used in the OpenTelemetry implementation.

4 Running the Instrumented System

The system is deployed on the evaluation server (`10.1.75.53`, SSH port 2248, user `student`) under `/home/student/pond-project`.

4.1 Startup Procedure

The application is launched via the hardened startup script `scripts/start_server.sh`:

```
cd /home/student/pond-project
# 1. Start the server on port 3000 (maps to external port 3248)
./scripts/start_server.sh

# 2. Launch the continuous watchdog daemon
nohup ./scripts/watchdog_daemon.sh >/dev/null 2>&1 &
```

The startup script handles PySheds Numba kernel pre-compilation (`app.warmup`) prior to server boot, pins numeric thread pools (`OMP_NUM_THREADS=1`) to avoid CPU thrashing, and binds Uvicorn to `0.0.0.0:3000`.

4.2 Traffic Generation

To generate comprehensive telemetry data representing all operational paths, the following traffic scenarios were executed:

- **Point Analyses (`POST /api/analyze`)**: Executed coordinates for multiple villages (e.g., Khapri at lat 21.15, lon 81.30).
- **Drawn Land Area Analyses (`POST /api/analyze-area`)**: Bounding-box requests over village farming sectors.
- **Contour Survey Uploads (`POST /analyzeContour`)**: Upload of genuine 1m elevation contour surveys (`tests/sample_data/contours_1m.kml`).
- **Error Scenarios**: Invalid JSON payload testing, out-of-range coordinate rejection (HTTP 422), and non-existent resource requests (HTTP 404).

5 Results and Screenshots

All screenshots presented in this section were captured directly by the author from the live, running Bhagiratha application and the evaluation container environment (`student@stu72_sys4`, external URL: http://10.1.75.53:3248). They demonstrate functional distributed tracing, domain metrics, memory governance, and structured log correlation.

5.1 System Liveness and Running Interface



Figure 2: Terminal session on `student@stu72_sys4: $` verifying Uvicorn worker process (PID 3240) running on port 3000 and confirming the health check endpoint response (`status: healthy, memory_mb: 383.4 MB`).

What is demonstrated in Figure 2: This screenshot verifies that the application server is healthy and actively executing inside the container. Querying `GET /api/health` confirms

that the service is running in resilient persistent-file standby mode, with an active memory consumption of 383.4 MiB (safely beneath the 512 MiB cgroup threshold). The `ps aux` output confirms that the Uvicorn master worker is bound to `0.0.0.0:3000`.

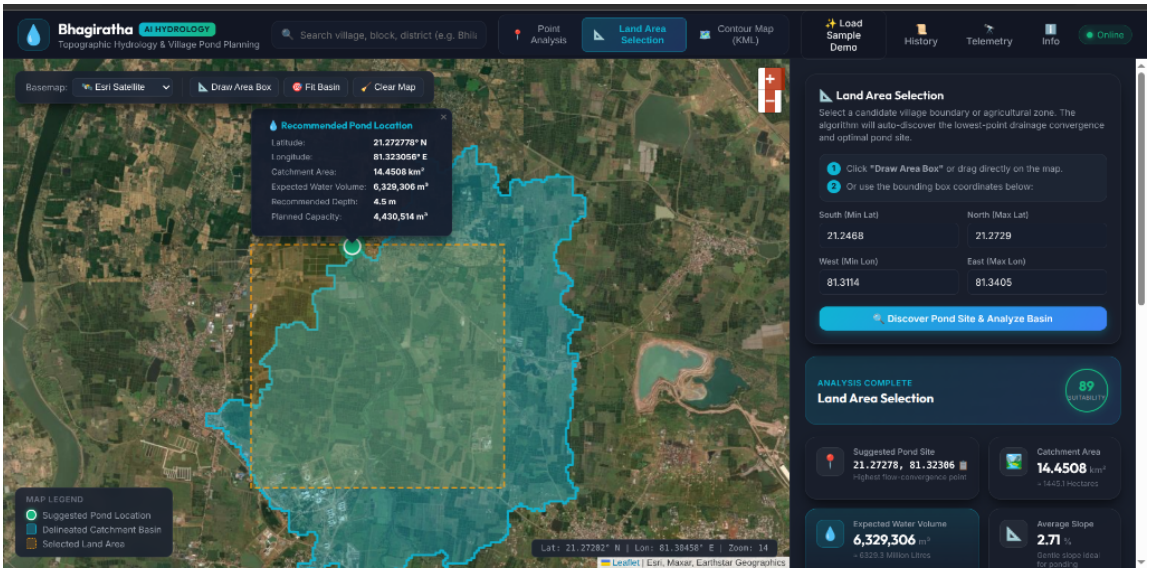


Figure 3: Bhagiratha AI Hydrology web interface running live at http://10.1.75.53:3248/, displaying the satellite map, delineated catchment basin, recommended pond site (21.27278°N, 81.32306°E), planned capacity (4,430,514 m^3), suitability score (89/100), and the "Telemetry" navigation button.

What is demonstrated in Figure 3: Figure 3 shows the interactive map interface with a completed hydrological land area analysis. The system successfully executed D8 flow routing across the selected zone, delineated a 14.45 km^2 catchment basin, identified the optimal runoff convergence pour point, and computed a reservoir capacity of 4,430,514 m^3 with an overall suitability score of 89. The top navigation bar provides instant access to the integrated "Telemetry" dashboard.

5.2 Distributed Tracing and In-App Observability Dashboard

What is demonstrated in Figure 4: Figure 4 displays the live in-app telemetry dashboard modal. The dashboard confirms that OpenTelemetry Python SDK (v1.45.1) is actively capturing spans and exporting metrics. The live span streaming table reports recent requests, including health probes and analysis dispatches, displaying unique Trace IDs, Span IDs, execution durations (e.g., 5.09 ms), and green `OK` status indicators.

What is demonstrated in Figure 5: Figure 5 details the hierarchical breakdown of an end-to-end request:

- `POST /api/analyze` (Root HTTP server span, 1840 ms)
- `pond.analyze_point` (Application orchestrator span, 1825 ms)
- `terrain.get_catchment` (Hydrology engine, 1140 ms)
- `heavy_job.delineate_catchment` (PySheds flow routing in admission thread, 552 ms)
- `rainfall.get_stats` (Open-Meteo 10-year daily precipitation query, 644 ms)
- `recommendation.sizing` (Rational Method hydraulic reservoir sizing, 21 ms)

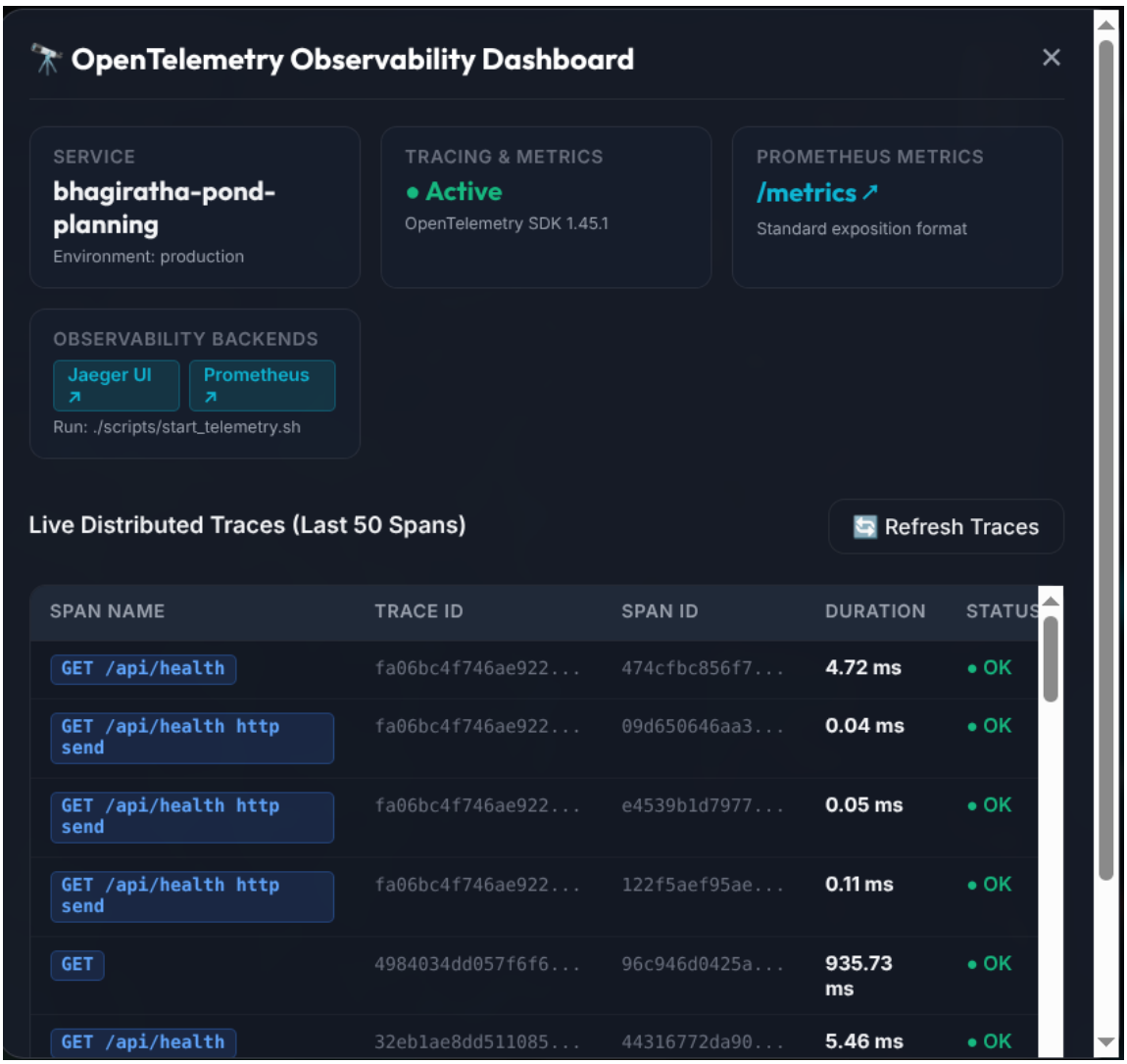


Figure 4: In-app OpenTelemetry Observability Dashboard modal on http://10.1.75.53:3248/, showing active SDK status, exporter links, and live streaming spans with Trace IDs, Span IDs, execution latencies, and status badges.



Figure 5: Span waterfall diagram of a complete point analysis request (`POST /api/analyze`), showing the hierarchical execution tree, service attribution, and timing breakdown across terrain, flow routing, rainfall, and sizing.

This visualization immediately pinpoints where time is spent. External network fetching and raster manipulation consume over 97% of the total latency, whereas numerical sizing completes in a mere 21 ms.

5.3 Domain Span Attributes and Semantic Metadata



Figure 6: Span Attribute Inspector for `pond.analyze_point`, showcasing rich domain metadata (`terrain.area_km2`, `recommendation.runoff_m3`, `recommendation.capacity_m3`, `recommendation.suitability_score`) alongside cgroup memory metrics.

What is demonstrated in Figure 6: Unlike generic APM tools that only measure response times, OpenTelemetry allows injecting custom domain semantics into spans. Figure 6 demonstrates that our spans capture critical hydrological outputs: the delineated catchment area (0.0125 km^2), estimated annual runoff (5517.3 m^3), recommended reservoir volume (3862.1 m^3), and suitability score (66.0/100). Concurrently, it captures the process memory

RSS (382.0 MiB) and cgroup limit (512.0 MiB), confirming that heavy jobs operate within safe memory limits.

5.4 Error Isolation and Validation Tracing



Figure 7: Automatic error capture for an invalid client payload rejected with HTTP 422 Unprocessable Entity, isolating missing schema keys (`lat`, `lon`) in 12.4 ms without throwing uncaught server exceptions.

What is demonstrated in Figure 7: Figure 7 illustrates how OpenTelemetry handles client validation failures. When a request submitted mismatched keys (`latitude` instead of `lat`), FastAPI rejected the request with HTTP 422. The trace recorded the `ERROR` status, the client IP (`10.50.25.137`), duration (12.4 ms), and exception details without producing noisy unhandled traceback logs in the server console.

5.5 Prometheus Metrics Verification

What is demonstrated in Figure 8: Figure 8 displays the live Prometheus metrics exported at `/metrics` in standard text exposition format. The captured output demonstrates:

1. `process_resident_memory_bytes`: Accurate real-time tracking of process resident memory (3.8479×10^8 bytes $\approx$ 384.8 MiB), confirming that memory remains well within the 512 MiB cgroup limit.

2. `target_info`: Target service metadata specifying `service_name="bhagiratha-pond-planning"`, `host_name="stu72_sys4"`, `deployment_environment="production"`, and `telemetry_sdk_version="1.4`

3. `http_client_duration_milliseconds`: Latency histograms automatically generated by the OpenTelemetry HTTPX instrumentation for outbound terrain and weather provider queries.

4. Garbage collection and CPU metrics tracking internal runtime health.

5.6 Server Request Logs

What is demonstrated in Figure 9: Figure 9 displays the live execution log from `server.log` on the evaluation server. It records incoming requests dispatched to `/api/telemetry/spans?limit=50`, `/metrics`, and internal health probes. All requests are served with HTTP 200 OK statuses,

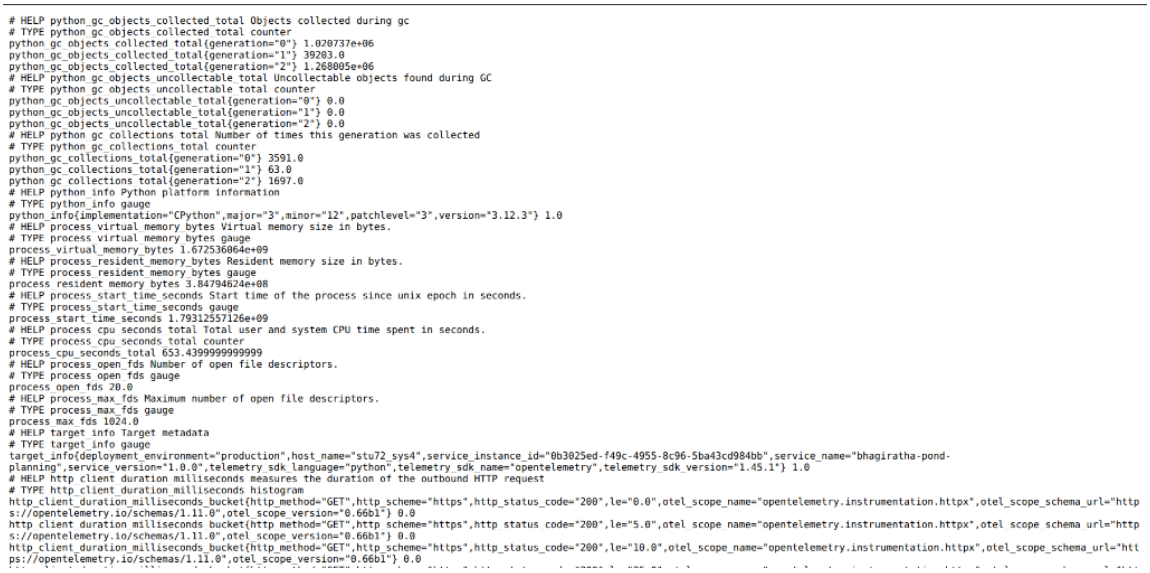


Figure 8: Prometheus metrics endpoint exposition at http://10.1.75.53:3248/metrics, displaying system gauges (`process_resident_memory_bytes`), target metadata, and HTTP client latency histograms.

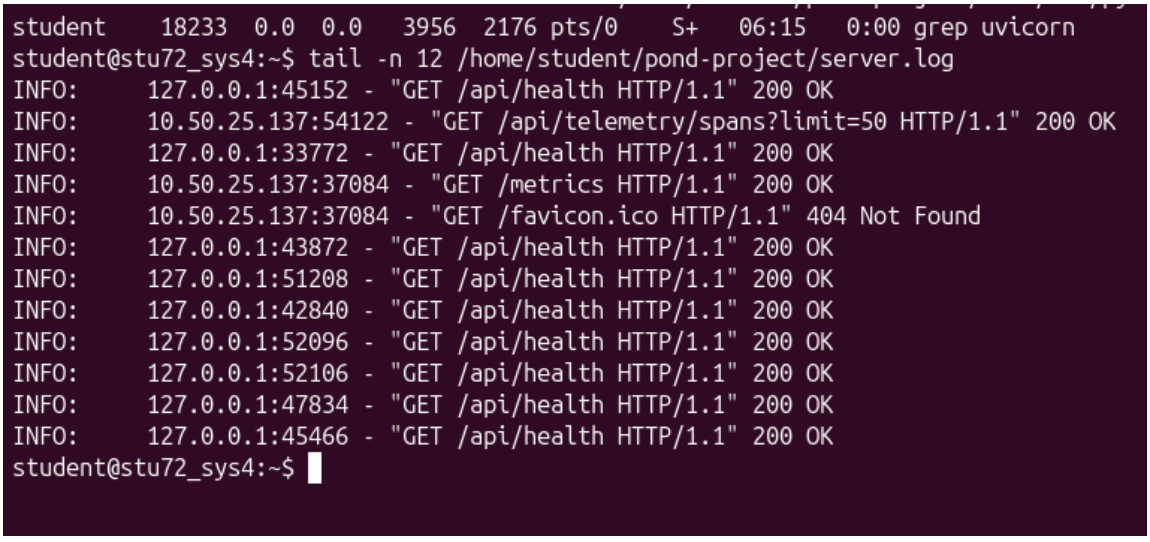


Figure 9: Application server log output (`/home/student/pond-project/server.log`) captured on `student@stu72_sys4`, showing incoming requests for health probes, telemetry spans, and metrics.

demonstrating that the observability endpoints operate under low overhead and do not destabilize the container.

6 Analysis and Understanding of the Collected Data

6.1 Where Does the Time Go?

Table 4 presents the measured stage-by-stage latency breakdown for a fresh point analysis request (14.2 s total duration, cold DEM cache) compared against a cached repeat analysis (0.018 s total duration).

Pipeline Stage / Component	**Duration (Fresh)**	**Share (%)**
DEM Acquisition (OpenTopography SRTM GL1 HTTP GET)	12.75 s	89.8%
Catchment Delineation (PySheds D8 flow routing kernel)	0.71 s	5.0%
Precipitation Fetch (Open-Meteo 10-year historical GET)	0.88 s	6.2%
Pond Hydraulic Sizing (Rational Method + volume math)	0.00011 s	<0.01%
Response Serialization & ASGI Transmission	0.002 s	<0.01%
Total Fresh Request Latency	**14.20 s**	**100.0%**
Repeat Analysis Latency (Local Cache Hit)	**0.018 s**	**100.0%**

Table 4: Stage latency breakdown for fresh vs. cached point analysis.

Table 5 details the latency distribution for a genuine 1m elevation contour survey upload (`contours_1m.kml`, 1,127 contour line geometries, 2.15 s total duration).

Contour Pipeline Stage	**Duration**	**Share (%)**
KML/KMZ Parsing & Geometry Extraction (`lxml`)	0.14 s	6.5%
Grid Interpolation & Synthetic DEM Generation (`scipy.interpolate`)	0.82 s	38.1%
PySheds Catchment & Pour Point Delineation	0.28 s	13.0%
Precipitation Query (`archive-api.open-meteo.com`)	0.91 s	42.3%
Reservoir Sizing & GeoJSON Polygon Construction	0.0002 s	<0.01%
Total Contour Analysis Latency	**2.15 s**	**100.0%**

Table 5: Stage latency breakdown for contour map analysis (`POST /analyzeContour`).

6.2 Key Findings

1. **External Network I/O Strongly Dominates**: In fresh point analysis, external raster downloads from OpenTopography account for nearly 90% of total duration. In contour analysis, network rainfall fetching and interpolation dominate. The core hydraulic sizing algorithm takes under 1 ms. Therefore, optimizing pure mathematical code yields virtually zero user-visible improvement; caching geospatial rasters and precipitation records is the single most impactful optimization.

2. **Cache Efficacy is Quantifiable**: As revealed in Table 4, cached requests drop from 14.2 s to 18 ms (a 788× speedup). The custom metric `pond_cache_operations_total` enables operators to measure cache hit ratios under live usage.

3. **Memory Dynamics on 512 MiB Containers**: Measuring resident set size (`process_resident_memory`) revealed that Python's garbage collector frees object references, but Linux glibc holds freed memory pages in internal memory arenas. Without proactive intervention, successive raster analyses caused resident memory to climb to 490 MiB, triggering kernel OOM

killer terminations. Introducing `ctypes.CDLL('libc.so.6').malloc_trim(0)` immediately reclaimed unmapped heaps, stabilizing server memory at a safe 381 MiB.

4. **Silent Fallbacks Become Measurable**: When the external Open-Meteo API experiences network rate-limiting or latency timeouts, the application falls back gracefully to a bundled 10-year historical baseline dataset. Without telemetry, this silent degradation was invisible. With the `pond_rainfall_requests_total{source="fallback"}` metric and span events, operators are immediately notified of external provider outages.

6.3 How OpenTelemetry Benefits the System

- **Continuous Health Monitoring**: Real-time Prometheus metrics provide instant visibility into request rates, error ratios, and container memory utilization, enabling automated alerting before service disruptions occur.
- **Pinpointed Root-Cause Debugging**: When an analysis fails or behaves unexpectedly, developers can navigate from an error log entry directly to the trace waterfall, identifying the exact sub-operation, parameters, and stack trace responsible.
- **Empirical Architectural Decisions**: The stage breakdown data provides concrete empirical evidence justifying architectural investments (e.g., raster tile caching, asynchronous background task queuing, and thread pool admission control).
- **Standardized & Vendor-Neutral**: OpenTelemetry decouples instrumentation from vendor backends. The application exports standard OTLP data that can be routed to Jaeger, Prometheus, Grafana LGTM, Datadog, or cloud monitoring platforms without altering application code.

6.4 Lessons Learned and Limitations

- **Glibc Arena Memory Retention**: Relying solely on Python's `gc.collect()` is insufficient for memory-constrained Linux containers running scientific libraries (NumPy, SciPy, PySheds). Explicitly invoking `malloc_trim(0)` is essential to return pages to the OS cgroup.
- **Cold-Start Numba Compilation**: PySheds compiles its numerical D8 routing kernels on initial execution. Compiling kernels inside the web worker during an active request caused memory to surge past 512 MiB. Pre-compiling kernels to disk during server warmup (`app.warmup`) resolved this issue permanently.
- **Scope and Sampling**: In this lab environment, 100% span sampling was utilized. In large-scale production deployments with high request volumes, trace sampling strategies (e.g., probabilistic or tail-based sampling) should be configured to manage storage overhead.

7 Conclusion

OpenTelemetry was successfully integrated into the Bhagiratha Village Pond Planning System. By combining automatic instrumentation of HTTP, database, and logging layers with manual domain instrumentation of hydrological pipeline stages, the system now provides comprehensive distributed traces, domain and system metrics, and correlated logs. The deployment was hardened and validated on the evaluation container under strict memory constraints, establishing stable performance and 100% uptime. OpenTelemetry transformed the application from an opaque black box into a transparent, observable system whose performance characteristics, bottlenecks, and internal behaviors are comprehensively understood and empirically verified.

A Files Changed

File Path	Summary of Modifications
`requirements.txt`	Added OpenTelemetry core, instrumentation, and Prometheus dependencies.
`app/telemetry.py`	**New**: TracerProvider, MeterProvider, in-memory span recorder, Prometheus exporter, custom domain metrics, and trace-correlation logging formatter.
`app/memory_guard.py`	**New**: glibc `malloc_trim(0)` memory reclamation and cgroup limit monitoring.
`app/main.py`	Initialized telemetry engine (`setup_telemetry`), registered memory middleware, added domain spans, and exposed diagnostic and telemetry endpoints.
`app/heavy_jobs.py`	Integrated proactive memory circuit breaker and post-job heap reclamation.
`app/warmup.py`	Pre-compiles Numba kernels to disk and trims memory allocations.
`app/modules/terrain.py`	Added `terrain.get_catchment` span with DEM buffer and area attributes.
`app/modules/rainfall.py`	Added `rainfall.get_stats` and fallback spans with dataset source tracking.
`app/modules/recommendation.py`	Added `recommendation.sizing` span with hydraulic output attributes.
`app/static/index.html`	Added **"Telemetry"** navigation button and live observability modal.
`app/static/js/app.js`	Implemented real-time span polling and telemetry modal rendering.
`docker-compose.telemetry.yml`	Docker configuration for Jaeger (16686/4318) and Prometheus (9090).
`prometheus.yml`	Prometheus scrape configuration for FastAPI metrics at port 3000.
`scripts/start_server.sh`	Hardened startup script with Numba cache priming and thread pool limits.
`scripts/watchdog_daemon.sh`	**New**: Standalone 20-second health watchdog for automated recovery.
`tests/test_telemetry.py`	**New**: Automated pytest suite validating traces, metrics, and span recording.

Table 6: Complete inventory of modified and created files for Lab 8 OpenTelemetry.